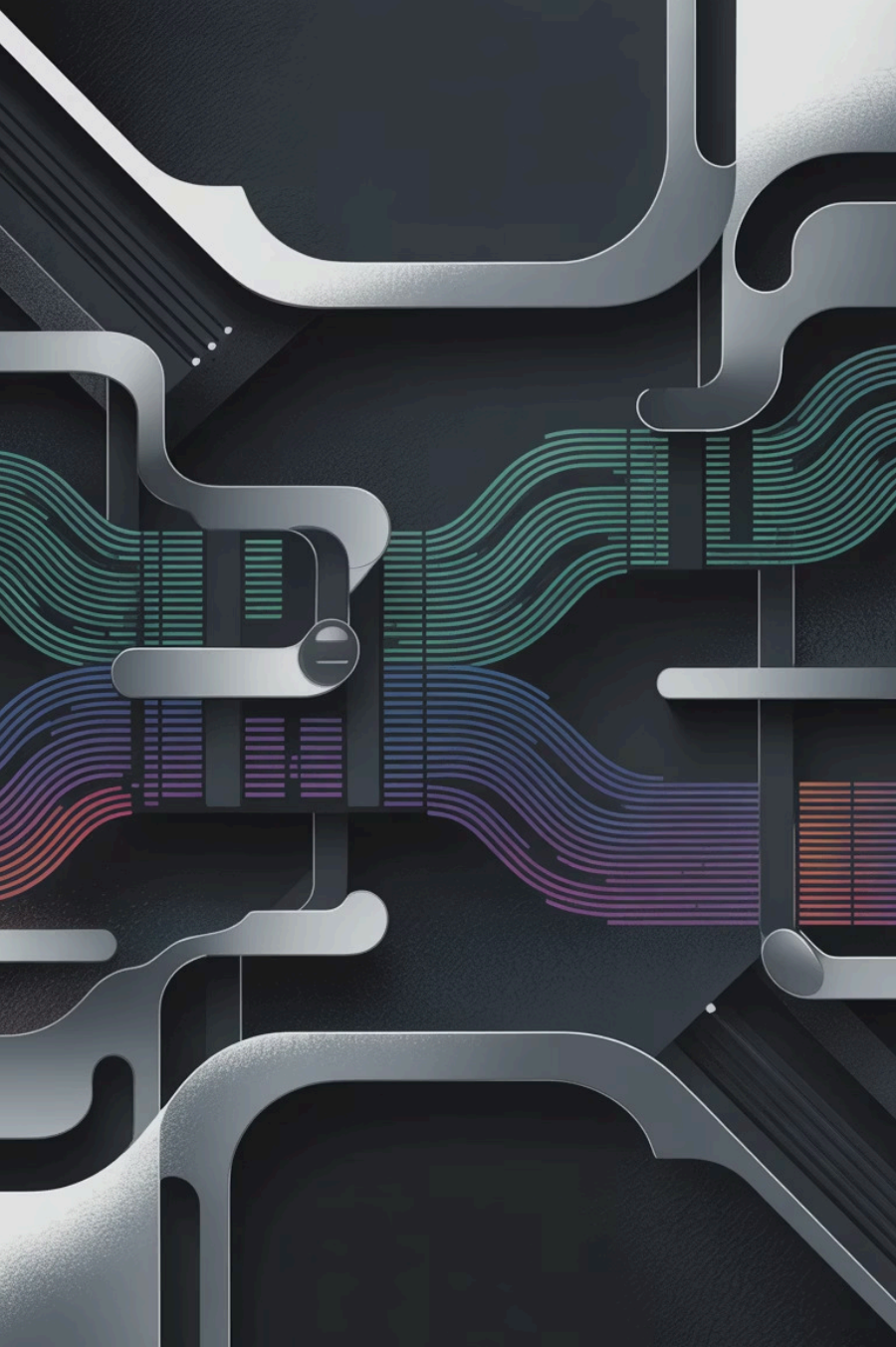




Module 01 - Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

What's On This Week's Agenda?

Over this week, you'll master the critical foundation of modern AI engineering: preparing data for large language models. This isn't just about moving data around—it's about building reliable, scalable pipelines that transform raw information into LLM-ready assets.

By the end of this week, you'll understand how to extract data from diverse sources, transform it into clean and validated datasets, and load it into formats that power cutting-edge AI applications.

What You'll Master

- Python-based data extraction from SQL databases and REST APIs
- Data transformation techniques including cleaning, joining, and validation
- LLM-specific data formatting and preparation
- Schema validation and data profiling for quality assurance



Why Data Engineering Matters for LLMs

Large language models are only as good as the data they're trained or fine-tuned on. Poor data quality leads to hallucinations, biased outputs, and unreliable AI systems. As an AI engineer, your ability to build robust data pipelines directly impacts the performance and trustworthiness of the models you deploy.

Modern LLM applications require data engineers who understand both traditional ETL principles and the unique requirements of machine learning workflows. This week bridges that gap, giving you the practical skills to succeed in this emerging field.

The Modern LLM Data Pipeline



Extract

Pull data from SQL databases, APIs, and other sources



Transform

Clean, join, and structure data for AI consumption



Validate

Ensure data quality through schema checks and profiling



Load

Format and store data for LLM training and fine-tuning

This pipeline represents the core workflow you'll build throughout the week. Each stage plays a critical role in ensuring your LLM receives high-quality, properly formatted training data.

Today's Learning Objectives

01

Understand the end-to-end data pipeline

See how extraction, transformation, validation, profiling, and loading connect

03

Identify quality assurance checkpoints

Discover where validation and profiling prevent downstream issues

02

Recognize LLM-specific data requirements

Learn what makes data "LLM-ready" and why it matters

04

Build mental models for this week's work

Create a framework for understanding each day's lessons in context



The Big Picture: From Raw Data to LLM-Ready Assets

Most organizations have data scattered across SQL databases, third-party APIs, CSV files, and other sources. This data is rarely in a format that LLMs can consume directly. Your role as an AI engineer is to build the bridge between these raw data sources and the structured, validated formats required for model training and fine-tuning.

Throughout this week, we'll walk through each step of this transformation, building practical skills with Python, SQL, and modern data engineering tools.

Stage 1: Data Extraction



Pulling Data from Multiple Sources

Extraction is about connecting to data sources and retrieving the information you need. In modern AI engineering, this typically means working with SQL databases and REST APIs using Python.

You'll learn to write efficient queries, handle API pagination, manage authentication, and deal with rate limits. These skills form the foundation of every data pipeline you'll build.

Key techniques: SQL queries with Python, API authentication, error handling, and connection pooling.

Common Data Sources for LLM Pipelines



SQL Databases

Relational databases like PostgreSQL and MySQL store structured business data. You'll use libraries like `psycopg2` or `SQLAlchemy` to query and extract this data efficiently.



REST APIs

Third-party services expose data through HTTP endpoints. You'll master the `requests` library to pull data from APIs, handling pagination, rate limits, and authentication tokens.



File Storage

CSV, JSON, and Parquet files often contain training data or exports from other systems. Python's `pandas` library makes it easy to read these formats into memory for processing.



Data Streams

Real-time data from message queues or event streams can feed continuously updating LLM applications. Advanced pipelines may incorporate Kafka or similar streaming platforms.

Extraction Example: SQL with Python

Here's what a basic extraction workflow looks like in practice. This example shows how to connect to a PostgreSQL database and extract customer feedback data for sentiment analysis training:

```
import psycopg
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine("postgresql+psycopg://analyst@localhost/customer_db")
...
# Extract feedback data
query = """
SELECT feedback_id, customer_id, feedback_text,
       rating, created_at
FROM customer_feedback
WHERE created_at > '2024-01-01'
"""
...
df = pd.read_sql_query(query, engine)
...
print(f"Extracted {len(df)} feedback records")
```

Extraction Example: REST API with Python

APIs require different handling than databases. You need to manage authentication, handle pagination for large datasets, and gracefully deal with rate limits. Here's a practical example:

```
import requests
import time

def fetch_api_data(endpoint, api_key):
    headers = {"Authorization": f"Bearer {api_key}"}
    all_data = []
    page = 1

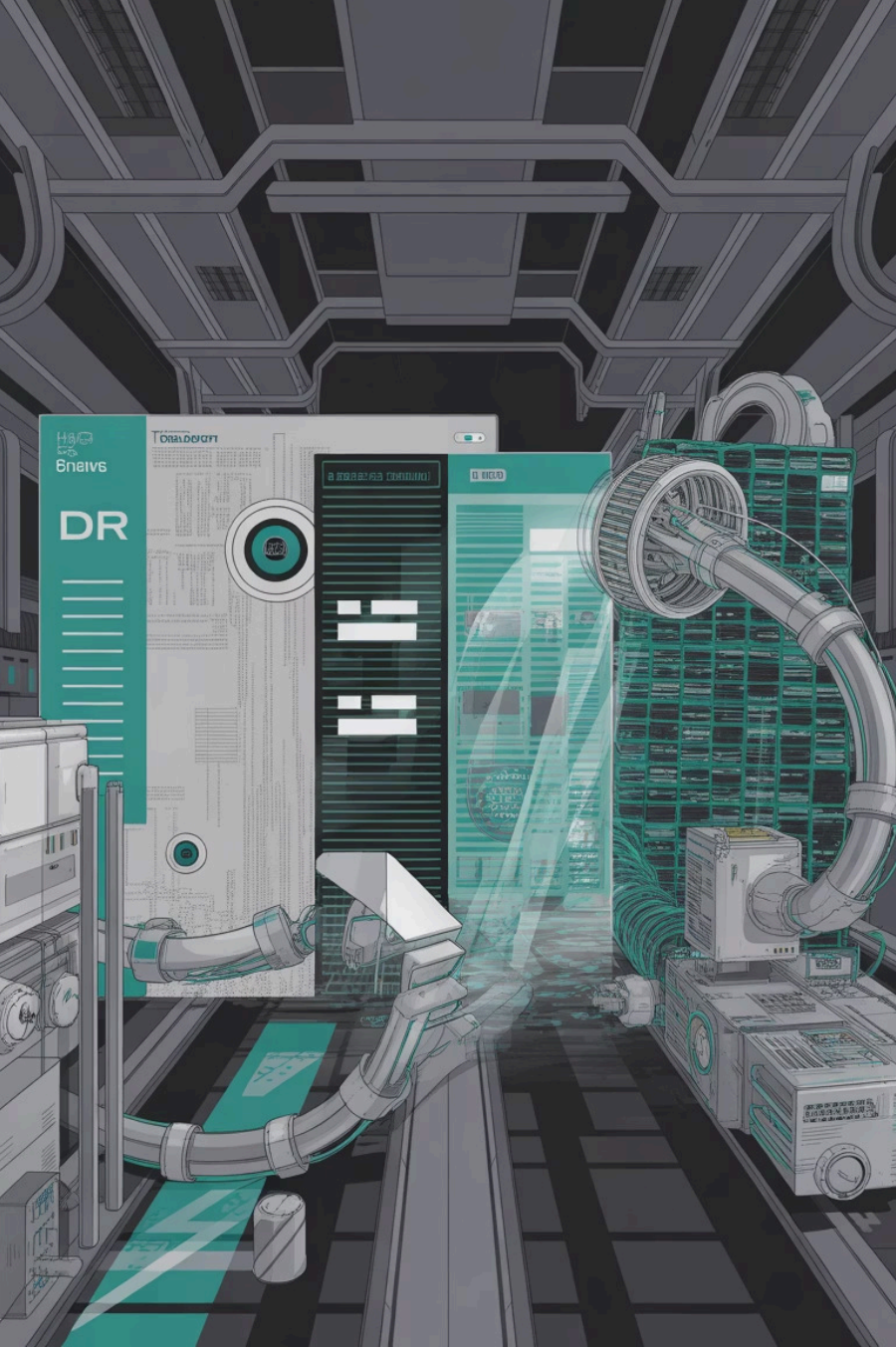
    while True:
        response = requests.get(
            f"{endpoint}?page={page}",
            headers=headers,
            timeout=(5, 30)
        )
        response.raise_for_status()

        if response.status_code == 429: # Rate limit
            time.sleep(60)
            continue

        data = response.json()
        if not data:
            break

        all_data.extend(data)
        page += 1

    return all_data
```

Stage 2: Data Transformation

Raw data is messy. It contains duplicates, missing values, inconsistent formats, and irrelevant information. Transformation is where you turn this messy data into clean, structured datasets ready for LLM consumption.

This stage involves cleaning operations like removing null values and duplicates, joining data from multiple sources to create richer contexts, normalizing text formats, and restructuring data to match your target schema. Good transformation practices directly impact model performance.

Core Transformation Techniques

1 Data Cleaning

Remove or impute missing values, eliminate duplicates, fix data type inconsistencies, and filter out irrelevant records. Cleaning ensures your LLM trains on accurate information.

2 Data Joining

Combine datasets from multiple sources using keys like customer IDs or product codes. Joins create richer training examples by adding context from related tables.

3 Text Normalization

Standardize text by handling case sensitivity, removing special characters, fixing encoding issues, and normalizing whitespace. Consistent formatting improves model training efficiency.

4 Feature Engineering

Create new fields that enhance training data quality, such as extracting timestamps into separate date components or calculating derived metrics that provide additional context.

Transformation in Action

Let's see how transformation cleans and structures the customer feedback data we extracted earlier. This example demonstrates several key techniques working together:

```
import pandas as pd

# Clean the data
df = df.dropna(subset=['feedback_text']) # Remove null text
df = df.drop_duplicates(subset=['feedback_id']) # Remove dupes
df['feedback_text'] = df['feedback_text'].str.strip() # Trim whitespace

# Join with customer data for context
customer_df = pd.read_sql_query(
    "SELECT customer_id, account_type, tenure_months FROM customers",
    conn
)
df = df.merge(customer_df, on='customer_id', how='left')
df['account_type'] = df['account_type'].fillna('unknown') # feedback with no matching customer

# Create useful features
df['feedback_length'] = df['feedback_text'].str.len()
df['is_negative'] = df['rating'] <= 3

print(f"Cleaned dataset: {len(df)} records with {len(df.columns)} features")
```

Why Transformation Matters for LLMs

Impact on Model Quality

Poorly transformed data leads to models that learn incorrect patterns, produce inconsistent outputs, or fail to generalize to new examples. Clean, well-structured data helps models learn faster and perform better in production.

LLMs are particularly sensitive to inconsistencies in text formatting, missing context, and unbalanced datasets. Your transformation choices directly influence whether the model will be reliable or problematic.





Stage 3: Schema Validation

Before moving data further down the pipeline, you need to verify it meets your quality standards. Schema validation ensures that every field has the correct data type, required fields are present, values fall within expected ranges, and the data structure matches your specifications.

Think of validation as quality gates in your pipeline. It's far better to catch issues early than to discover problems after spending hours or days training a model on bad data.

What Schema Validation Checks



Data Types

Verify that numeric fields contain numbers, dates are in correct formats, and text fields don't contain unexpected characters or encoding issues.



Value Ranges

Check that numeric values fall within expected bounds, like ratings between 1-5, or text length within reasonable limits for your use case.



Required Fields

Ensure critical columns have no null values. For LLM training, you typically need complete text and label pairs without missing data.



Referential Integrity

Verify that foreign keys match existing records and relationships between tables are maintained correctly after joins.

Implementing Validation with Pandera

Pandera is a Python library that makes schema validation straightforward and readable. Here's how to define and enforce a schema for our customer feedback data:

```
import pandera.pandas as pa

# Define schema
schema = pa.DataFrameSchema({
    "feedback_id": pa.Column(int, nullable=False, unique=True),
    "customer_id": pa.Column(int, nullable=False),
    "feedback_text": pa.Column(str, pa.Check.str_length(min_value=10), nullable=False),
    "rating": pa.Column(int, pa.Check.in_range(1, 5)),
    "created_at": pa.Column(pd.DatetimeTZDtype(tz="UTC"), coerce=True),
    "account_type": pa.Column(str, pa.Check.isin(['free', 'premium', 'enterprise', 'unknown']))
})

# Validate dataframe
try:
    validated_df = schema.validate(df)
    print("✓ Schema validation passed")
except pa.errors.SchemaError as e:
    print(f"✗ Validation failed: {e}")
```

Stage 4: Data Profiling

Understanding Your Data's Characteristics

Profiling goes beyond validation to provide deep insights into your dataset's statistical properties and patterns. You'll examine distributions of values, identify outliers and anomalies, detect correlations between fields, and spot potential biases in your training data.

Profiling helps you make informed decisions about how to prepare data for LLMs. For instance, you might discover that 90% of your training examples are positive sentiment, indicating a need for better balance.



Key Profiling Metrics for LLM Data



Distribution Analysis

Examine how values are spread across your dataset. Are text lengths consistent? Are labels balanced? Understanding distributions helps identify potential training issues.



Outlier Detection

Find records that deviate significantly from typical patterns. Outliers might represent data quality issues or rare but important edge cases for your model.



Duplication Checks

Identify exact or near-duplicate records that could cause models to overfit. Near-duplicates are particularly problematic in training sets.



Bias Detection

Look for imbalances that could lead to biased model behavior, such as underrepresented categories or skewed label distributions.

Profiling with Pandas

Python's pandas library provides powerful profiling capabilities. Here's how to generate comprehensive statistics about your dataset:

```
# Basic statistics
print(df.describe())

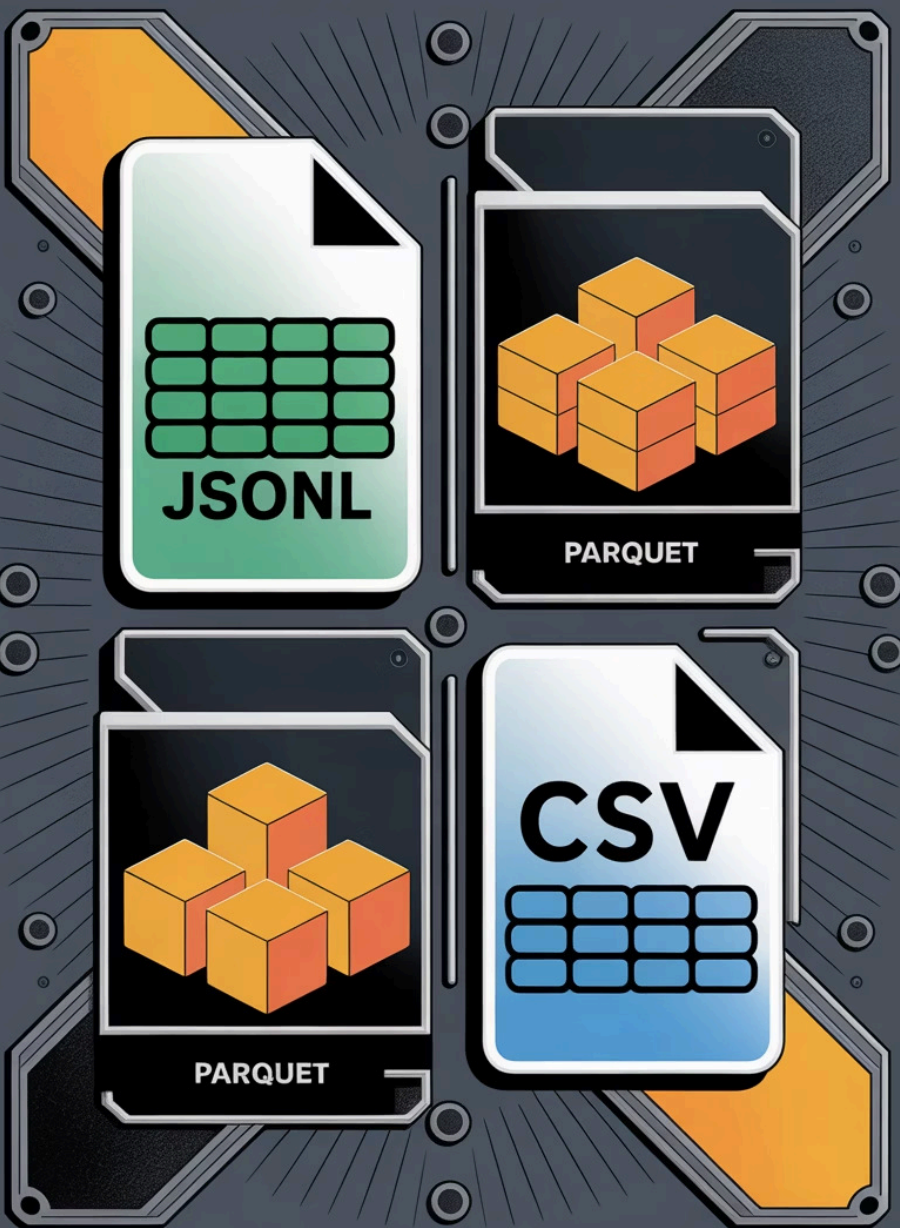
# Check for missing values
print(df.isnull().sum())

# Examine value distributions
print(df['rating'].value_counts())
print(df['account_type'].value_counts(normalize=True))

# Analyze text length distribution
df['text_length'] = df['feedback_text'].str.len()
print(df['text_length'].describe())

# Check for duplicates
duplicates = df.duplicated(subset=['feedback_text']).sum()
print(f"Found {duplicates} duplicate feedback entries")

# Detect outliers using IQR method
Q1 = df['text_length'].quantile(0.25)
Q3 = df['text_length'].quantile(0.75)
IQR = Q3 - Q1
outliers = ((df['text_length'] < (Q1 - 1.5 * IQR)) |
            (df['text_length'] > (Q3 + 1.5 * IQR))).sum()
print(f"Found {outliers} text length outliers")
```



Stage 5: Loading LLM-Ready Formats

After extraction, transformation, validation, and profiling, your data is clean and verified. The final stage is loading it into formats optimized for LLM training and fine-tuning workflows. Different LLM frameworks and training pipelines expect different formats, and choosing the right one impacts both performance and compatibility.

This stage involves selecting appropriate file formats, structuring data according to framework requirements, optimizing for efficient loading during training, and organizing datasets for train/validation/test splits.

Common LLM Data Formats

1 JSONL (JSON Lines)

One JSON object per line, making it easy to stream large datasets. Popular for fine-tuning GPT models and other transformer architectures. Each line typically contains input text and corresponding labels or completions.

2 Parquet

Columnar storage format that's highly efficient for large datasets. Supports compression and fast querying. Excellent for storing embeddings and features alongside text data.

3 HuggingFace Datasets

Specialized format used by the HuggingFace ecosystem. Optimized for NLP tasks with built-in support for streaming, caching, and efficient batching during training.

Creating JSONL for Fine-Tuning

JSONL is the most common format for fine-tuning LLMs. Here's how to convert your cleaned feedback data into the format expected by OpenAI's fine-tuning API:

```
import json

def create_training_jsonl(df, output_file):
    """Convert feedback data to OpenAI fine-tuning format"""
    with open(output_file, 'w', encoding='utf-8') as f:
        for _, row in df.iterrows():
            # Create training example
            example = {
                "messages": [
                    {
                        "role": "system",
                        "content": "You are a customer feedback analyzer."
                    },
                    {
                        "role": "user",
                        "content": row['feedback_text']
                    },
                    {
                        "role": "assistant",
                        "content": f"Rating: {row['rating']}/5. "
                                f"Sentiment: {'positive' if row['rating'] > 3 else 'negative'}"
                    }
                ]
            }
            f.write(json.dumps(example, ensure_ascii=False) + '\n')

create_training_jsonl(df, 'feedback_training.jsonl')
```

The Complete Pipeline in Context



Now you can see how all five stages work together. Data flows from sources through each transformation, with validation and profiling acting as quality gates. Each stage depends on the previous one being done correctly. Skip validation, and you might train on corrupt data. Rush transformation, and your model learns from inconsistent formats.

Real-World Pipeline Example

Consider a company building a customer support chatbot. They need to fine-tune an LLM on historical support tickets. The pipeline extracts tickets from their ticketing system database and customer data from a CRM API, transforms by cleaning text and joining customer context, validates that all tickets have proper categories and resolutions, profiles to ensure balanced representation of issue types, and loads into JSONL format for GPT fine-tuning.

This real-world scenario demonstrates why each stage matters and how they solve actual business problems.



Common Pitfalls in LLM Data Pipelines

1 Skipping Validation

Training on unvalidated data leads to mysterious model failures hours or days later. Always validate before moving to the next stage. The time investment pays off exponentially.

2 Ignoring Data Distributions

Imbalanced or skewed training data creates biased models. Profile your data to catch these issues early, before training begins.

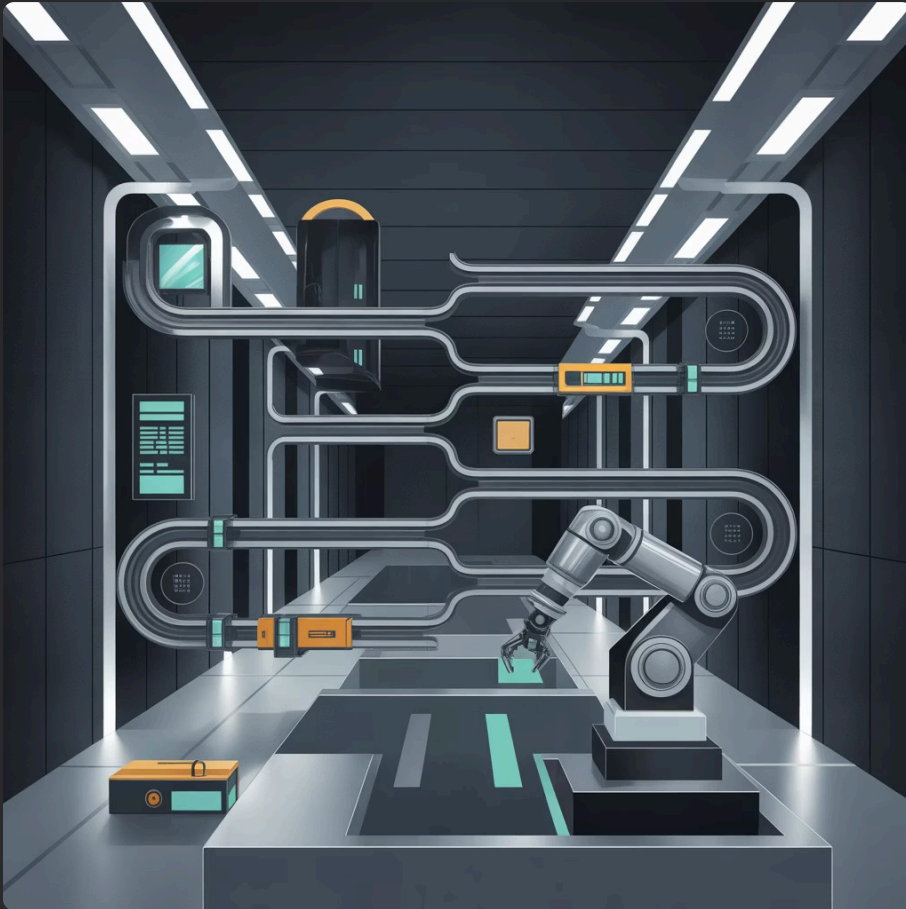
3 Inconsistent Text Encoding

Mixed encodings cause garbled text in training data. Standardize encoding during transformation to prevent subtle corruption that breaks model performance.

4 Missing Context

LLMs perform better with rich context. Don't extract just the minimum fields—join relevant metadata that helps the model understand the situation better.

Quality Gates: When to Stop and Fix Issues



Implementing Pipeline Checkpoints

Don't push bad data forward. Establish clear quality thresholds at each stage: after extraction, check that you received expected record counts; after transformation, verify no unexpected nulls were introduced; after validation, ensure error rates are below tolerance; and after profiling, confirm distributions match expectations.

Automated checks save time and prevent expensive mistakes downstream.

Data Quality Metrics That Matter

95%

Minimum Completeness

At least 95% of required fields should be non-null after transformation

<2%

Duplication Rate

Keep duplicates under 2% to prevent overfitting during model training

100%

Schema Conformance

Every record must pass schema validation before loading into training format

±20%

Balance Threshold

No single category should represent more than 20% deviation from expected distribution

Best Practices for Production Pipelines

1 Implement Logging

Log every stage of your pipeline with timestamps, record counts, and any warnings or errors. This makes debugging and monitoring much easier in production environments.

2 Make Pipelines Idempotent

Design pipelines so they can be safely re-run without creating duplicates or corrupting data. Use unique identifiers and check for existing data before inserting.

3 Add Configuration Files

Store database credentials, API keys, and pipeline parameters in configuration files separate from your code. This makes pipelines portable and secure.

4 Version Your Data

Track which version of the pipeline produced each dataset. This helps reproduce results and debug issues when model performance changes unexpectedly.

Error Handling Strategies

Building Resilient Pipelines

Production pipelines face API timeouts, database connection failures, and malformed data. Robust error handling prevents these issues from crashing your entire workflow.

Implement retry logic with exponential backoff for temporary failures. Catch and log specific exceptions rather than using broad try-except blocks. Create fallback mechanisms for critical operations, and alert humans when manual intervention is needed.

```
import time
from functools import wraps

def retry_on_failure(max_attempts=3, delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except requests.exceptions.RequestException as e:
                    if attempt == max_attempts - 1:
                        raise
                    wait_time = delay * (2 ** attempt)
                    print(f"Attempt {attempt + 1} failed: {e}")
                    print(f"Retrying in {wait_time} seconds...")
                    time.sleep(wait_time)
            return wrapper
        return decorator

@retry_on_failure(max_attempts=3)
def fetch_api_data(url):
    response = requests.get(url)
    response.raise_for_status()
    return response.json()
```

Monitoring Pipeline Performance

1

Pipeline Success Rate

Track what percentage of pipeline runs complete successfully without errors or data quality failures

2

Processing Time

Monitor how long each stage takes to identify bottlenecks and optimize performance

3

Data Quality Score

Measure overall data quality using metrics like completeness, validity, and consistency

Regular monitoring helps you spot issues before they impact model training. Set up automated alerts for metrics that fall outside acceptable ranges.



Thanks